

TRACING STRUKTUR DATA

1. Tracing Proses Tree

- Representasi Heap dalam Array

Heap direpresentasikan menggunakan ArrayList. Hubungan antara parent dan child dihitung menggunakan rumus:

left child = $2i + 1$

right child = $2i + 2$

parent = $(i - 1) / 2$

Kondisi Awal Sebelum Heapify:

{'A1', 'A2', 'A3', 'A4', 'A5', 'A6', 'B1', 'B2', 'B3', 'B4', 'B5', 'C1', 'C2', 'C3', 'C4', 'C5', 'D1', 'D2', 'D3', 'D4', 'D5', 'E1', 'E2', 'E3', 'E4', 'E5'}

- Manual Tracing Proses Heapify

Jumlah checkpoint adalah 26, sehingga proses heapify dimulai dari index:

$n / 2 - 1 = 26 / 2 - 1 = 12$

Artinya, proses bubbleDown() dilakukan mulai dari index 12 sampai index 0.

Langkah	Index	Node yang Dicek	Proses
1	12	C2(6)	Tidak terjadi swap karena child tidak lebih besar.
2	11	C1(2)	Swap dengan E4(4), karena E4 memiliki danger score lebih tinggi.
3	10	B5(8)	Tidak terjadi swap.
4	9	B4(4)	Swap dengan D5(7).
5	8	B3(6)	Tidak terjadi swap.
6	7	B2(5)	Swap dengan C5(8).
7	6	B1(3)	Swap dengan C3(9).
8	5	A6(90)	Tidak terjadi swap karena A6 sudah memiliki nilai sangat besar.
9	4	A5(6)	Swap dengan B5(8).
10	3	A4(4)	Swap dengan C5(8).
11	2	A3(2)	Swap dengan A6(90), lalu bubble down dilanjutkan.
12	1	A2(3)	Swap dengan B5(8), lalu bubble down

			dilanjutkan.
13	0	A1(2)	Swap dengan A6(90), lalu bubble down dilanjutkan sampai posisi heap valid.

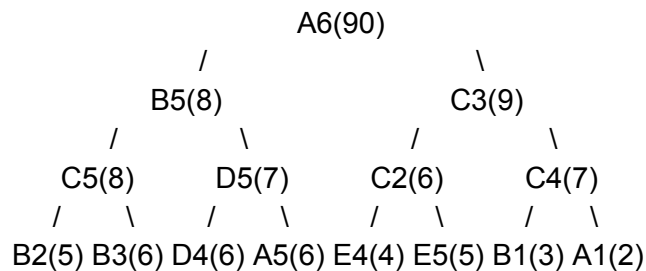
- Hasil Akhir Heap

Isi heap dalam bentuk array setelah proses heapify selesai:

{A6(90), B5(8), C3(9), C5(8), D5(7), C2(6), C4(7), B2(5), B3(6), D4(6), A5(6), E4(4), E5(5), B1(3), A1(2), A4(4), D1(2), D2(3), D3(5), A2(3), B4(4), E1(1), E2(2), E3(3), C1(2), A3(2)}

Dengan root heap: A6(90)

- Bentuk Tree Setelah Heapify



Meskipun C3(9) lebih besar daripada B5(8), struktur ini tetap valid karena keduanya adalah child dari A6(90). Dalam heap, yang penting adalah hubungan parent-child, bukan urutan antar sibling.

- Manual Tracing Operasi Peek

Operasi peek() digunakan untuk melihat checkpoint dengan prioritas tertinggi tanpa menghapusnya dari heap.

Karena root heap berada pada index 0, maka:

peek() = heap[0] = A6(90)

Langkah	Index	Node yang Dicek
1	0	A6(90)

- Manual Tracing Operasi Top 5 Ranking

Untuk menampilkan ranking checkpoint paling berbahaya, program menggunakan method toSortedList(limit). Method ini membuat salinan heap, lalu melakukan proses extractMax() berulang kali.

Pada setiap proses extractMax():

1. Root heap diambil sebagai nilai maksimum.
2. Elemen terakhir dipindahkan ke root.
3. Proses bubbleDown() dilakukan untuk mengembalikan sifat Max Heap.
4. Nilai maksimum dimasukkan ke list hasil ranking.

Urutan	Root yang Diambil	Keterangan
1	A6(90)	Danger score terbesar, langsung keluar pertama.
2	C3(9)	Setelah A6 dihapus, C3 menjadi prioritas tertinggi.
3	B5(8)	Memiliki danger score 8 dan ID lebih kecil dibanding C5.
4	C5(8)	Danger score sama dengan B5, tetapi ID lebih besar, sehingga keluar setelah B5.
5	C4(7)	Setelah nilai 8 habis, checkpoint dengan nilai 7 menjadi prioritas berikutnya.

Maka hasil ranking top 5 adalah:

1. A6(90)
2. C3(9)
3. B5(8)
4. C5(8)
5. C4(7)

Checkpoint C3 tetap masuk ke heap walaupun statusnya blocked = true, karena heap ini dibuat berdasarkan seluruh checkpoint dan tidak memfilter checkpoint yang terblokir.

2. Tracing Proses Graph

- Tracing BFS dari Spawn Gate (A1)

BFS melakukan eksplorasi secara merata dengan FIFO, tanpa mempertimbangkan bobot terrain, mengabaikan adanya wall dan blocked node.

Langkah	Dequeue	Asal Node	Enqueue
1	A1	A1	A2, A6, B1
2	A2	A1	A3, B2
3	A6	A1	E2
4	B1	A1	C1

5	A3	A2	A4, B3
6	B2	A2	C2
7	E2	A6	D2, E1, E3
8	C1	B1	D1
9	A4	A3	A5, B4
10	B3	A3	- (C3 = WALL)
11	C2	B2	-
12	D2	C2	D3
13	E1	E2	-
14	E3	E2	E4
15	D1	C1	-
16	A5	A4	B5
17	B4	A4	C4
18	D3	D2	D4
19	E4	E3	E5
20	B5	A5	C5
21	C4	B4	-
22	D4	D3	D5
23	E5	E4	-
24	C5	B5	-
25	D5	D4	-

- **Tracing A***

A* memakai $f(n) = g(n) + h(n)$.

Dengan heuristic:

Langkah	Dequeue	g	h	f	Asal Node	Enqueue
1	A1	0	8	8	-	A2, A6, B1

2	A2	1	7	8	A1	A3, B2
3	B1	1	7	8	A1	B2, C1
4	B2	2	6	8	B1	B3, C2
5	C1	2	6	8	B1	D1
6	A3	3	6	9	A2	A4, B3
7	D1	4	5	9	C1	D2, E1
8	B3	4	5	9	A3	B4
9	D2	5	4	9	D1	D3, E2
10	E2	6	3	9	D2	A6, E3
11	E2	6	3	9	D2	A6, E3
12	E3	7	2	9	E2	E4
13	D3	7	3	10	D2	D4
14	D4	8	2	10	D3	C4, D5
15	E5	10	0	10	E4	-

Jalur: A1 - B1 - C1 - D1 - D2 - E2- E3- E4 - E5

Total cost: 10

Tanpa heuristic:

Langkah	Dequeue	g	h	f	Asal Node	Enqueue
1	A1	0	0	0	-	A2, A6, B1
2	A2	1	0	1	A1	A3, B2
3	B1	1	0	1	A1	B2, C1
4	B2	2	0	2	B1	B3, C2
5	C1	2	0	2	B1	D1
6	A3	3	0	3	A2	A4, B3
7	D1	4	0	4	C1	D2, E1

8	B3	4	0	4	A3	B4
9	D2	5	0	5	D1	D3, E2
10	E1	5	0	5	D1	-
11	C2	6	0	6	B2	-
12	A4	6	0	6	A3	A5
13	E2	6	0	6	D2	A6, E3
14	B4	7	0	7	B3	B5, C4
15	D3	7	0	7	D2	D4
16	E3	7	0	7	E2	E4
17	D4	8	0	8	D3	D5
18	E4	9	0	9	E3	E5
19	E5	10	0	10	E4	-

3. Tracing Edge Case

- Checkpoint Awal Tidak Valid

Contoh kasus:

Start = Z9

Goal = E5

Langkah	Kondisi	Respon Program
1	User memasukkan start Z9 dan goal E5.	Program menerima input.
2	Program mencari checkpoint Z9.	Checkpoint tidak ditemukan.
3	Start tidak valid.	Pencarian tidak dijalankan.
4	Program mengembalikan hasil.	Jalur tidak ditemukan.

Kesimpulan: Jika checkpoint awal tidak tersedia di map, algoritma tidak dijalankan karena tidak ada titik mulai yang valid untuk melakukan pencarian.

- Checkpoint Tujuan Tidak Valid

Contoh kasus:

Start = A1

Goal = Z9

Langkah	Kondisi	Respon Program
---------	---------	----------------

1	User memasukkan start A1 dan goal Z9.	Program menerima input.
2	Program memeriksa A1.	Checkpoint awal valid.
3	Program mencari Z9.	Checkpoint tujuan tidak ditemukan.
4	Goal tidak valid.	Pencarian dihentikan.
5	Program mengembalikan hasil.	Jalur tidak ditemukan.

Kesimpulan: Jika checkpoint tujuan tidak tersedia, program tidak memiliki target pencarian sehingga jalur tidak dapat ditemukan.

- Checkpoint Tujuan Terblokir

Contoh kasus:

Start = A1

Goal = C3

Pada dataset, C3 merupakan checkpoint yang terblokir.

Langkah	Kondisi	Respon Program
1	User memasukkan start A1 dan goal C3.	Program menerima input.
2	Program memeriksa A1.	Checkpoint awal valid.
3	Program memeriksa C3.	Checkpoint ditemukan.
4	Status C3 adalah blocked.	Goal dianggap tidak dapat diakses.
5	Program menghentikan pencarian.	Jalur tidak ditemukan.

Kesimpulan: Checkpoint tujuan yang blocked tidak dapat dijadikan target akhir karena area tersebut dianggap tidak dapat diakses.

- Route Bertipe WALL

Contoh kasus:

Route B3 -> C3 memiliki terrain WALL

Langkah	Kondisi	Respon Program
1	Program memeriksa route dari checkpoint saat ini.	Route-route tetangga dicek.

2	Program menemukan route bertipe WALL.	Route dianggap tidak dapat dilewati.
3	Route WALL diabaikan.	Program tidak memasukkan tujuan route tersebut sebagai kandidat.
4	Program lanjut ke route lain.	Hanya route valid yang diproses.

Kesimpulan: Route bertipe WALL tetap tersimpan sebagai bagian dari map, tetapi tidak digunakan sebagai jalur valid dalam BFS maupun A*.

- Jalur Lebih Pendek Tetapi Cost Lebih Mahal

Contoh kasus:

Jalur 1: A1 -> A6 -> E2 -> E3 -> E4 -> E5

Jalur 2: A1 -> B1 -> C1 -> D1 -> E1 -> E2 -> E3 -> E4 -> E5

Langkah	Kondisi	Respon Program
1	Program menemukan beberapa kemungkinan jalur.	Semua kandidat jalur dievaluasi.
2	Jalur 1 memiliki jumlah checkpoint lebih sedikit.	Program tetap menghitung total cost.
3	Jalur 1 melewati terrain berat seperti LAVA.	Total cost menjadi lebih besar.
4	Jalur 2 memiliki checkpoint lebih banyak.	Program tetap membandingkan cost.
5	Jalur 2 melewati terrain lebih ringan.	Total cost dapat menjadi lebih kecil.
6	Program memilih jalur dengan cost terkecil.	Jalur terbaik ditentukan dari total cost, bukan jumlah checkpoint.

Kesimpulan: Dalam weighted graph, jalur terbaik ditentukan berdasarkan total cost, bukan berdasarkan jumlah checkpoint yang dilewati. Karena itu, jalur yang lebih panjang bisa menjadi pilihan terbaik jika melewati terrain dengan bobot lebih ringan.

- Tidak Ada Jalur Valid ke Tujuan

Contoh kasus:

Start dan goal tersedia, tetapi semua jalur menuju goal terhalang WALL atau checkpoint blocked.

Langkah	Kondisi	Respon Program
---------	---------	----------------

1	Program memulai pencarian dari start.	Start valid.
2	Program memeriksa route yang tersedia.	Route valid diproses.
3	Route WALL ditemukan.	Route diabaikan.
4	Checkpoint blocked ditemukan.	Checkpoint diabaikan.
5	Semua kandidat habis diperiksa.	Goal tidak tercapai.
6	Program mengembalikan hasil.	Jalur tidak ditemukan.

Kesimpulan: Meskipun start dan goal tersedia di map, jalur tetap bisa tidak ditemukan jika semua kemungkinan route menuju tujuan tidak dapat dilewati.

- **Start dan Goal Sama**

Contoh kasus:

Start = A1

Goal = A1

Langkah	Kondisi	Respon Program
1	User memasukkan start dan goal yang sama.	Program menerima input.
2	Program memeriksa checkpoint tersebut.	Checkpoint valid dan tidak blocked.
3	Start sudah sama dengan goal.	Pencarian langsung selesai.
4	Program membentuk hasil jalur.	Jalur hanya berisi A1.
5	Program menghitung cost.	Total cost bernilai 0.

Kesimpulan: Jika start dan goal sama, jalur valid tetap ditemukan dengan total cost 0 karena karakter sudah berada di checkpoint tujuan.